CHAPTER 5

Register-Transfer Level (RTL) Design

5.1 INTRODUCTION

Previous chapters introduced methods to capture behavior and to implement that behavior as a digital circuit. Chapter 2 introduced a method to capture basic combinational behavior using equations or truth tables, and to implement that behavior as a circuit of two or more levels of gates. Chapter 3 introduced a method to capture basic sequential behavior using finite-state machines (FSMs), and to implement that behavior as a circuit having a register and combinational logic, which together are known as a controller. This chapter will focus on capturing even higher-level sequential behavior, using a high-level state machine (HLSM) whose inputs, outputs, state actions, and transition conditions can all involve higher-level data types like binary numbers or integers rather than just the Boolean type used in FSMs. To implement such behavior, this chapter will introduce a method to convert a high-level state machine into a circuit consisting of a controller connected to a datapath, which together are known as a ***processor***. The datapath is composed of datapath components defined in Chapter 4, including registers, adders, comparators, etc., custom connected such that the datapath can perform the specific operations defined by the high-level state machine. The controller sets the datapath's signals in each state such that the state's actions and transition conditions are carried out.

The above discussion uses the term "higher-level," which should be explained. Digital designers commonly distinguish among the different levels shown in Figure 5.1. The more complex the building blocks, the higher the level of *abstraction* that the designer deals with. Connecting transistors into circuits to build gates or other components is called ***transistor-level design***. Designing combinational or sequential circuits as in Chapters 2 and 3 involves circuits whose building blocks are primarily logic gates, and is thus called ***logic-level design***. Designing processors involves circuits whose building blocks are registers and other datapath components, and involves transferring data from registers, through other datapath components like adders, and back to registers. Such design is thus called *register-transfer level design* or ***RTL design***—which is the focus of this chapter. In the 1970s and 1980s, most digital design practice occurred at the logic level. Today, most practice is at the register-transfer level. Improving tools continue to move design practice to higher levels. Higher levels deal with fewer and higher-

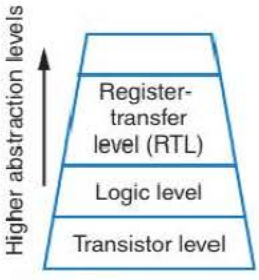


Figure 5.1 Levels of digital design.

complexity building blocks, and thus can enable design of higher-complexity circuits with less time and effort.

The term "microprocessor" became popular in the 1980s when programmable processors shrank from occupying many boards or chips down to occupying just a single chip. "Micro" refers to being small.

The name *processor* is best known from its use in the name *microprocessor*. A microprocessor is a *programmable* processor, which is a general predesigned processor created to carry out any desired computation (see Chapter 8). This chapter instead focuses on designing *custom processors*, which are processors each of whose design is specialized to implement one specific computation, like converting a Celsius number to Fahrenheit. As such, custom processors can be extremely small, low-power, and fast compared to programmable processors. Custom processors and programmable processors often coexist in digital systems. For example, a TV set-top box may use a programmable processor to carry out most of the functions related to changing channels, controlling volume, etc., but may use custom processors to very quickly decompress the video data that is streaming into the system and to quickly display that data on the TV screen.

RTL design begins by capturing desired behavior. A formalism for capturing RTL behavior is a high-level state machine.

▶ 5.2 HIGH-LEVEL STATE MACHINES

Some behaviors are too complex to capture using just an equation, truth table, or FSM. Consider capturing the behavior of a custom processor for a soda machine dispenser that dispenses a soda when enough money has been deposited into the machine. A block diagram of the processor system is shown in Figure 5.2. A coin detector provides the processor with a 1-bit input `c` that becomes `1` for one clock cycle when a coin is detected, and an 8-bit input *a* indicates the value in cents of the inserted coin, such as 25 cents (`00011001`) or 10 cents (`00001010`). Another 8-bit input *s* indicates the cost of a soda, such as 60 cents (`00111100`), which can be set by the machine owner. When the processor has detected that the total value of deposited coins equals or exceeds the cost of a soda (e.g., 25 + 25 + 10 >= 60), the processor should set an output bit `d` to `1` for one clock cycle, causing a soda to be dispensed (this machine has only one type of soda, and does not give change). Assume that the value *a* persists until the next coin is deposited, and that many clock cycles (e.g., thousands) occur between successive coins being deposited.

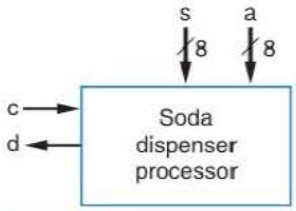


Figure 5.2 Soda dispenser block symbol.

An FSM is not sufficient for capturing the data aspects of this system's behavior. An FSM can only have Boolean (i.e., single-bit) inputs, not an 8-bit data input representing a binary number. An FSM has no convenient way of keeping track of the total data value of coins deposited so far. An FSM can only perform Boolean operations, not the data addition operation (e.g., 25 + 10) required to keep track of the total value of coins deposited.

A ***high-level state machine*** (***HLSM***) extends FSMs with the data features needed to capture more complex behaviors, including:

- *multibit data inputs and outputs* rather than just single bits,
- *local storage*, and
- *arithmetic operations* like add and compare, rather than just Boolean operations.

This chapter will use HLSMs whose local storage is loaded on rising clock edges. Also, each local storage item and multibit input or output is assumed to be unsigned, unless specifically denoted as "signed" (see Section 4.6).

Figure 5.3 shows an HLSM describing the behavior of the soda dispenser processor. The HLSM initially sets output `d` to `0` and sets a local storage item *tot* to 0. The HLSM then waits in state *Wait* to detect a coin being deposited. When detected, the HLSM goes to state *Add*, which adds the coin's value *a* to *tot*, after which the HLSM returns to *Wait*. If *tot*'s value is less than the cost *s* of a soda (*tot* < *s*), the HLSM continues to wait for more coins. Otherwise, the HLSM goes to state *Disp*, which sets `d` to `1` to dispense a soda, after which the HLSM returns to state *Init* to clear *tot* back to 0 and start over again.

As a reminder, this book usually uses a `Courier` *font for names and constants representing a bit, and italics for other names. So "d" and "0" represent bit values, while "tot" and "0" represent normal data items.*

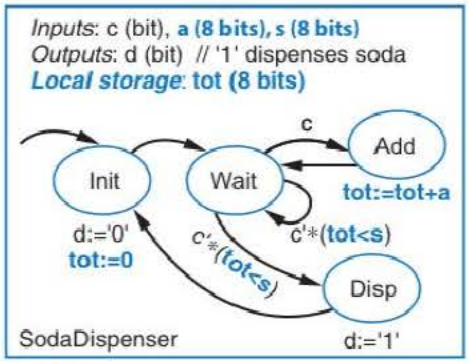


Figure 5.3 Soda dispenser high-level state machine with non-FSM constructs highlighted.

The state machine is *not* an FSM, because of reasons highlighted in the figure. One reason is because the state machine has inputs that are 8-bit types, whereas FSMs only allow inputs and outputs of Boolean types (a single bit each). Another reason is because the state machine declares local storage *tot* to store intermediate data, whereas FSMs don't allow local data storage—the only "stored" item in an FSM is the state itself. A third reason is because the state actions and transition conditions involve data operations like *tot* := 0 (remember that *tot* is 8-bits wide), *tot* < *s*, and *tot* := *tot* + *a* (where the "+" is addition, not OR), whereas an FSM allows only Boolean operations like AND or OR.

This chapter will use the following conventions for HLSMs, also used for FSMs:

- Each transition is implicitly ANDed with a rising clock edge.
- Any *bit* output not explicitly assigned a value in a state is implicitly assigned a `0`. Note: this convention does not apply for multibit outputs.

This chapter will also use the following conventions for HLSMs:

- To distinguish between a bit `0` or `1` and a binary number 0 or 1, HLSMs will surround bit values with single quotes as in `'0'` and `'1'`. In Figure 5.3, note that the bit output `d` is assigned the bit `'0'` while the multibit storage *tot* is assigned 0 (without quotes), which is the integer representation of the 8-bit binary number `00000000`. Being 8 bits wide, *tot* could be assigned the number 0, 1, 2, 3, ..., up to 255. In contrast, being a bit, `d` can only be assigned `'0'` or `'1'`. To assign a multibit item with a multibit constant, double quotes will be used, e.g., *tot* = `"00000000"`.
- To avoid confusion between arithmetic comparison and assignment, HLSMs will use "`==`" for comparison and "`:=`" for assignment. "=" will not be used for either. In Figure 5.3, note that assigning `d` is written as `d := '0'` rather than as `d = 0`, and assigning *tot* is written as *tot* := 0. If *tot* had to be compared to *s* for equality on a transition, such comparison would be written as *tot* == *s*, making clear that *tot* is

being compared with *s* and not being assigned the value of *s*. Comparing for less than or equal to would be written as *tot* <= *s*.

- Every HLSM multibit output is registered (see Chapter 3). As such, every HLSM multibit output *X* must have a storage item *Xreg* declared that is the same width as *X*. Writing to *X* is accomplished by writing to *Xreg*; writes directly to *X* are not allowed. *Xreg* can be read; *X* cannot. If desired, a single-bit output `B` can be registered by declaring a local storage item `Breg`.

To aid in understanding, an HLSM can include text that describes some aspect of the HLSM. Such text is preceded by "//" and is known as a ***comment***. One comment appears in Figure 5.3 to describe the behavior of output `d`.

As was true for FSMs, capturing behavior as an HLSM can be challenging. As in Chapter 3 for FSMs, the capture process for HLSMs can be aided by first listing all possible states (with some transitions included if helpful), then creating all possible transitions leaving each state, and finally by mentally executing the HLSM and refining it if necessary. The following example illustrates creation of an HLSM for a simple system.

Example 5.1 Cycles-high counter

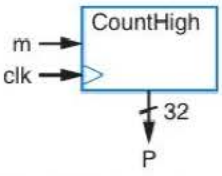


Figure 5.4 Cycles-high counter block diagram.

This example captures an HLSM for a system having a bit input `m` and a 32-bit output `P`. The system should output the total number of clock cycles for which the input `m` is `1`. For example, after powering on the system, if `m` is `0` for 20 cycles, then `1` for 15 cycles, then `0` for 12 cycles, and then `1` for 3 cycles, the system output at that point should be 18 (15 + 3). `P` connects to a display that converts the 32-bit number into a displayed integer. Such a cycles-high counter system might be useful to determine the total time that a car's brakes are applied, that a laser has been turned on, etc.

Figure 5.5(a) shows the first HLSM state *S_Clr* that initializes the output *P* to 0 by setting its storage item *Preg* to 0. *Preg* accomplishes the storage of the cycles-high count, and thus declaring another local storage item is not necessary. Figure 5.5(b) introduces a second state *S_Wt* that waits for `m` to be `1`; transitions are shown for this state. Finally, Figure 5.5(c) introduces a third state *S_Inc* that increments *Preg* once for each clock cycle that `m` is `1`; transitions are also shown for this state. The HLSM now has all possible transitions. Mentally executing the HLSM seems to validate that it correctly captures the desired behavior.

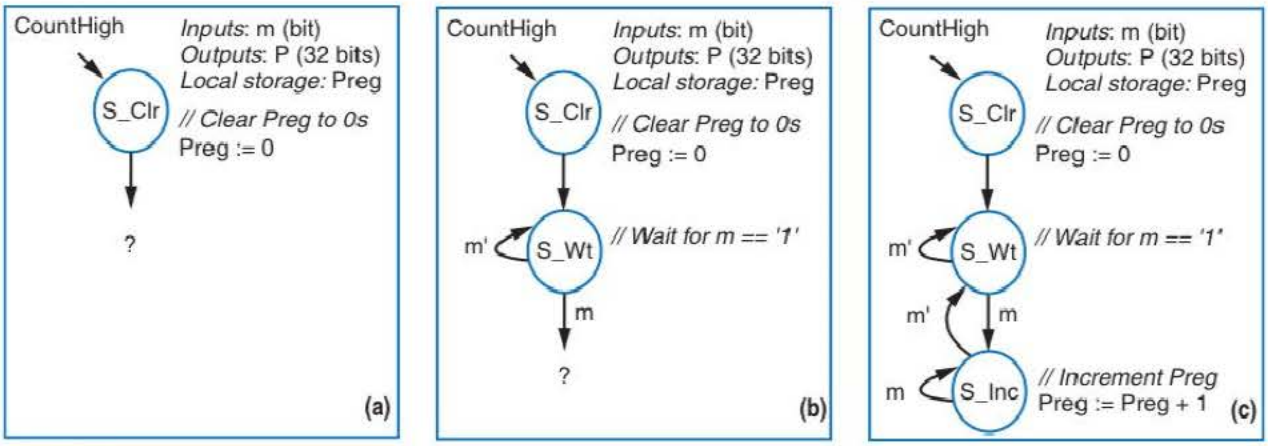


Figure 5.5 HLSM for cycles-high counter: (a) initial state, (b) waiting for `m` to be `1`, (c) incrementing *Preg* when `m` is `1`.

Example 5.2 Laser-based distance measurer

Many applications require accurately measuring the distance of an object from a known point. For example, road builders need to accurately determine the length of a stretch of road. Map makers need to accurately determine the locations and heights of hills and mountains and the sizes of lakes. A giant crane for constructing skyrise buildings needs to accurately determine the distance of the sliding crane arm from the base. In all of these applications, stringing out a tape measure to measure the distance is not very practical. A better method involves laser-based distance measurement.

In laser-based distance measurement, a laser is pointed at the object of interest. The laser is briefly turned on and a timer is started. The laser light, traveling at the speed of light, travels to the object and reflects back. A sensor detects the reflection of the laser light, causing the timer to stop. Knowing the time T taken by the light to travel to the object and back, and knowing that the speed of light is $3x10^8$ meters/second, the distance D can be computed easily by the equation: $2D = T$ seconds $* 3x10^8$ meters/second. Laser-based distance measurement is illustrated in Figure 5.6.

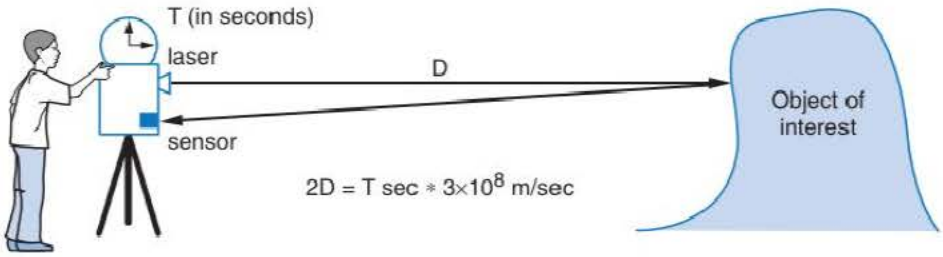


Figure 5.6 Laser-based distance measurement.

This example captures an HLSM to describe the behavior of a processor that controls a laser to compute distances up to 2000 meters. A block diagram of the system is shown in Figure 5.7. The system has a bit input `B`, which equals `1` when the user presses a button to start the measurement. Another bit input `S` comes from the sensor and is `1` when the reflected laser is detected. A bit output `L` controls the laser, turning the laser on when `L` is `1`. Finally, an N-bit output D indicates the distance in binary, in units of meters—a display converts that binary number into a decimal number and displays the results on an LCD for the user to read. D will have to be at least 11 bits, since 11 bits can represent the numbers 0 to 2047, and the system should measure distances up to 2000 meters. To be safe, we'll make D 16 bits.

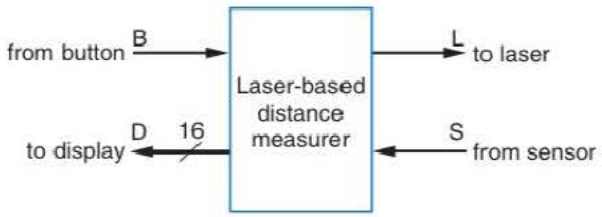


Figure 5.7 Block diagram of the laser-based distance measurer.

To facilitate the creation of the state machine, we first enumerate the sequence of events that would typically occur in the measurement system:

- The system powers on. Initially, the system's laser is off and the system outputs a distance of 0 meters.
- The system waits for the user to press button `B` to initiate measurement.
- After the button is pressed, the system should turn the laser on. We'll choose to leave the laser on for one clock cycle.

- After the laser is turned on, the system should wait for the sensor to detect the laser's reflection. Meanwhile, the system should count how many clock cycles occur from the time the laser was turned on until the reflection is sensed.
- After the reflection is sensed, the system should use the number of cycles that occurred since the laser was pulsed to compute the distance to the object of interest. The system should then return to waiting for the user to press the button again so that a new measurement can be taken.

The above sequence guides our construction of an HLSM. We begin with an initial state named *S0* as shown in Figure 5.8. *S0's* task is to ensure that when the system powers on, the system does not output an incorrect distance, and the system does not turn the laser on. Note that output *D* is not written directly, but rather the output's local storage *Dreg* is written instead. Recall that the assignment `L := '0'` assigns the bit `0` to the one-bit output `L`, whereas the assignment *Dreg* := 0 assigns the 16-bit binary number 0 (which is actually `0000000000000000`) to the 16-bit storage *Dreg*.

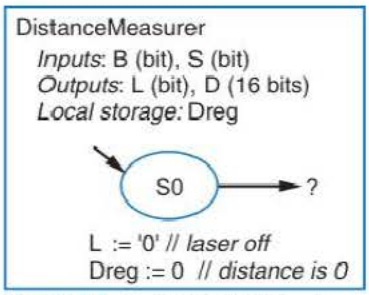


Figure 5.8 Partial HLSM for the distance measurer: Initialization.

After initialization, the measurement system waits for the user to press the button to initiate the measurement process. To perform the waiting, we add a state named *S1* as shown in Figure 5.9. The shown transitions cause the state machine to remain in state *S1* while `B` is `0` (i.e., while `B'` is true). Be aware that the `L` is implicitly assigned `0` in this state, because of the convention that any single-bit output not explicitly assigned in a state is implicitly assigned `0`. On the other hand, *Dreg*, corresponding to a local storage item, is unchanged in this state.

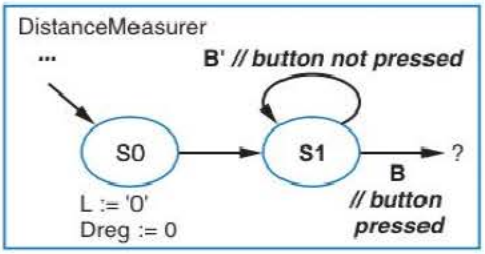


Figure 5.9 Distance measurer HLSM (cont.): Waiting for a button press.

When `B` becomes `1`, the laser should be turned on for one cycle. The HLSM should transition to a state that turns the laser on, followed by a state that turns the laser off. We'll call the laser-on state *S2* and the laser-off state *S3*. Figure 5.10 shows how *S2* and *S3* are introduced into the HLSM.

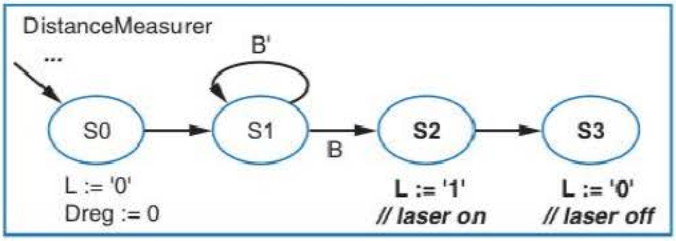


Figure 5.10 Distance measurer HLSM (cont.): Turning the laser on for one cycle.

In state *S3*, the HLSM should wait until the sensor detects the laser's reflection (`S`). The state machine remains in *S3* while `S` is `0`. The HLSM should meanwhile count the clock cycles between the laser being turned on and the laser's reflection being sensed. We can measure time by counting the number of clock cycles and multiplying that number by the clock period (*T* = cycles counted * clock period). Thus, we introduce a *local storage* item named *Dctr* to keep track of the cycles counted. The HLSM increments *Dctr* as long as the HLSM is waiting for the laser's reflection. (For simplicity, we ignore the possibility that no reflection is detected.) The HLSM must also initialize *Dctr* to 0 before

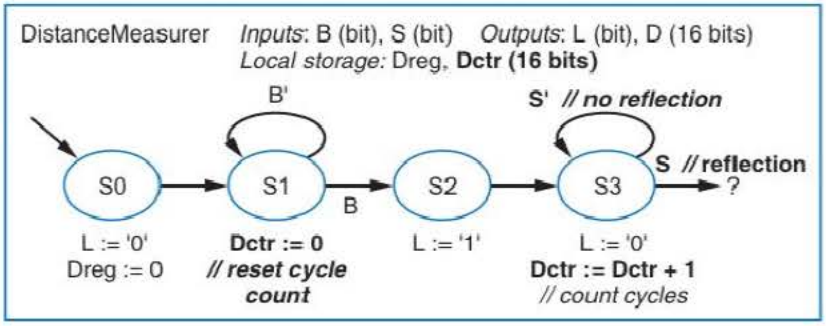


Figure 5.11
Distance measurer HLSM (cont.): Waiting for the laser reflection and counting clock cycles.

counting; such initialization can be added to state *S1*. With these modifications, the HLSM is seen in Figure 5.11.

Once the reflection is detected (`S=1`), the HLSM should compute the distance D that is being measured. Figure 5.6 shows that $2*D = T * 3x10^8$ m/sec. We know that the time T in seconds is *Dctr* * *clock period*. To simplify the calculation of D, assume the clock frequency is $3x10^8$ Hz (which is 300 MHz), so the clock period is $1 / (3x10^8)$ sec. Thus, $2*D = (Dctr / 3x10^8)$ sec $* 3x10^8$ meters/sec = *Dctr* meters, and so $D = (Dctr / 2)$ meters. We'll perform this calculation in a state named *S4*. The final HLSM is shown in Figure 5.12. All possible transistions from each state have already been included. A mental execution seems to confirm that the HLSM behaves as desired.

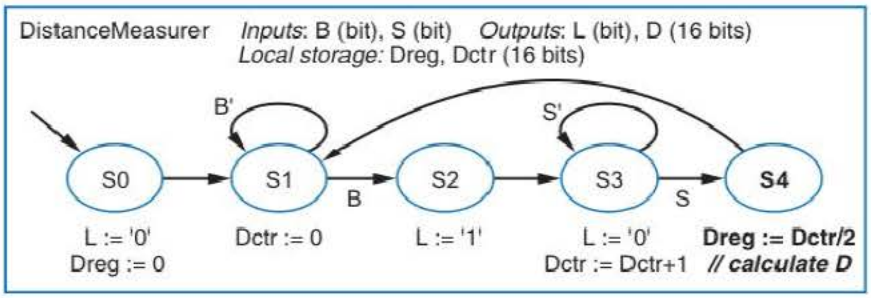


Figure 5.12
Completed HLSM for the distance measurer, including calculation of *D*.

A laser-based distance measurer could use a faster clock frequency to measure distance with a greater precision than 1 meter.

The HLSM described above is just one type of FSM extension. Dozens of extended FSM varieties exist. The particular variety of HLSM described above and used throughout this chapter is sometimes called an ***FSM with data*** or ***FSMD***. A different state machine variation that was previously popular was called ***algorithmic state machines***, or ***ASMs***. ASMs are similar to flowcharts, except that ASMs include a notion of a clock that enables transitions from one state to another (a traditional flowchart does not have an explicit clock concept). ASMs contain more "structure" than a state machine. A state machine can transition from any state to any other state, whereas an ASM restricts transitions in a way that causes the computation to look more like an algorithm—an ordered sequence of instructions. An ASM uses several types of boxes, including state boxes, condition boxes, and output boxes. ASMs typically also allowed local data storage and data operations. The advent of hardware description languages (see Chapter 9) seems to have largely replaced the use of ASMs, because hardware description languages

contain the constructs supporting algorithmic structure, and much more. We do not describe ASMs further.

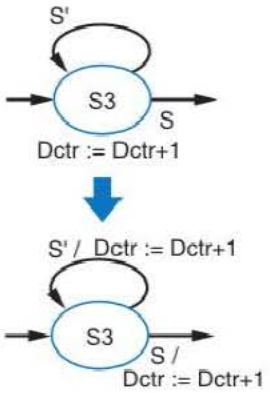


Figure 5.13 A storage update can be thought of as occuring on outgoing transitions.

In the HLSMs of this chapter, all writes to storage items in a state's actions are to storage items that are loaded on rising clock edges only. As such, writing a value to a storage item in a state's actions does not actually cause the storage item to be updated until the *next rising clock* edge. So the update occurs at the *end* of the state when the next rising clock edge causes a transition. For example, consider the state action "*Dctr* := *Dctr* + 1" in state *S3* of the distance measurer HLSM in Figure 5.12. When a clock edge causes a transition from *S2* to *S3*, *Dctr* will initially be 0. While in *S3* during that clock cycle, *Dctr* will still be 0. When the *next* clock edge arrives, *Dctr* will be updated to 1 (i.e., to *Dctr* + 1, meaning 0 + 1 = 1), and the HLSM will transition back to *S3* if `S` is `0`. *Dctr* will remain 1 until the next clock edge, when it will become *Dctr* + 1 = 1 + 1 = 2. A simple way to visualize a state's local storage updates is to consider those updates as occuring on each outgoing transition instead of in the state, as in Figure 5.13. Because local storage item updates occur at the end of a state when the next rising clock edge occurs, and because transitions are taken on that same clock edge, then a transition whose condition uses the storage item uses the non-updated value. A common mistake is to assume the transition uses the updated value.

For example, suppose a system waits for an input control bit `B` to become `1` and then sets an output `P` to `1` for 50 cycles. Rather than using 50 states to set `P` high, an HLSM can use a local storage item. Figure 5.14(a) shows an HLSM using local storage *Jreg*, for the case of setting the output `P` to `1` for 2 cycles rather than 50 so that the example is easier to understand. The diagram for that HLSM in Figure 5.14(b) shows that *S0*'s action of *Jreg* := 1 occurs at the same time as the transition to *S1* during the next clock edge. Then, even though *S1*'s action is *Jreg* := *Jreg* +1, *J* remains 1 for the duration of the clock cycle, and so the conditions on *S1*'s transitions with *Jreg*<2 will be comparing 1<2. On the next clock cycle, the transition back to *S1* will be taken and *Jreg* will simultaneously be updated to 2 (remember, you can consider the storage update as occuring on the outgoing transitions). During that second cycle in *S1*, the transition comparison will be 2<2, so on the next clock edge the transition to *S0* will be taken and *Jreg* will simultaneously be updated with 3 (that value of 3 will never be used, and at the end of state *S0*, *Jreg* will become 1 again).

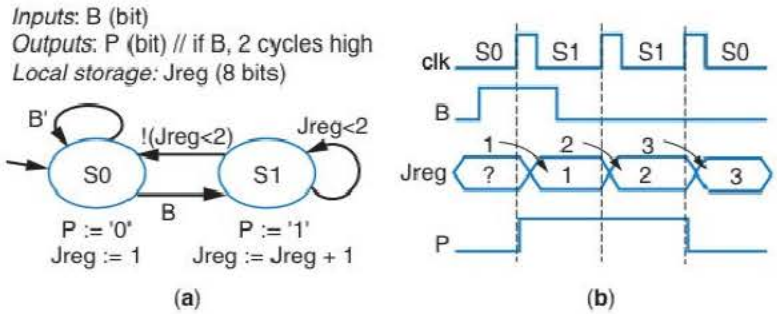


Figure 5.14 HLSM clocked storage update behavior: (a) HLSM with storage *Jreg*, (b) *S0*'s setting *Jreg* to *1* doesn't occur until the next clock edge, which is also when *S1* is entered. *S1*'s setting of *Jreg*:=*Jreg*+1 will set *Jreg* to 2, but not until the next edge, meaning *Jreg*<2 is false for the first *S1* state.

Because the updates of a state's actions occur at the next clock edge, the order in which local storage item actions are listed in a state does not matter—all the updates occur simultaneously at the end of the state. For example, swapping two storage items *A* and *B* in a state could be achieved by "*A* := *B*" followed by "*B* := *A*", or by "*B* := *A*" followed by "*A* := *B*." The results are identical in either listing of the actions—at the end of the state, *A* and *B* get updated with the previous values of *B* and *A*, respectively. Think of the updates being prepared in the state, but not actually occuring until the next clock. Figure 5.14(b) shows the updates that are prepared in each state, with arrows indicating when the updates actually occur.

► 5.3 RTL DESIGN PROCESS

RTL design follows a two-step process, as was the case for combinational design in Chapter 2 and for sequential design in Chapter 3. The first step is to *capture* the desired behavior, and the second step is to *convert* that behavior as a circuit. This chapter captures behavior using an HLSM; designers sometimes use other formalisms too. Converting an HLSM to a circuit is aided by the use of a standard processor architecture, similar to how converting an FSM to a circuit in Chapter 3 was aided by the use of a standard controller architecture consisting of a state register and combinational logic. A standard processor architecture is shown in Figure 5.15, consisting of a controller connected with a datapath. The datapath will have the ability to carry out each particular data operation present in the HLSM, by having the necessary datapath components (e.g., if the HLSM requires an addition and a comparison, then the datapath will include an adder and a comparator). The controller will set the control input signals of the various datapath components (e.g., the load control input of a register) to carry out the specific actions of each particular state and to transition to appropriate next states based on the control output signals of those datapath components.

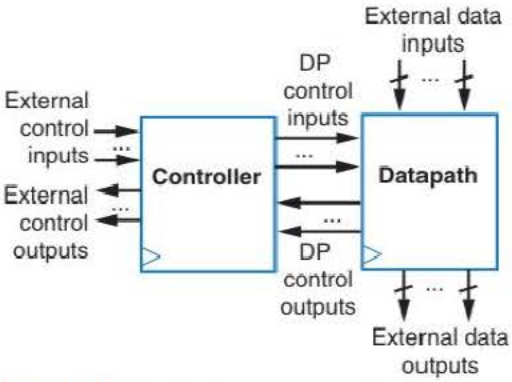


Figure 5.15 Standard processor architecture: controller/datapath pair.

To help understand the controller/datapath pair in the standard processor architecture, consider the cycles-high counter from Example 5.1, which is revisited in Figure 5.16. Figure 5.16(a) shows the system block diagram and describes the system's desired behavior. As this chapter's convention is to always register data outputs, the figure already shows a register *Preg* connected to data output *P*. Figure 5.16(b) shows the desired behavior captured as an HLSM created in the earlier example. The behavior requires that the computation "*Preg* := *Preg* + 1" be performed. As such, Figure 5.16(c) shows an adder whose two inputs are *Preg* and a constant 1, and whose output is connected to *Preg*. Thus, computing "*Preg* := *Preg* + 1" can be achieved simply by setting *Preg*'s `ld` control input to `1`. Furthermore, the required computation "*Preg* := 0" can be achieved by setting *Preg*'s `clr` control input to `1`. Thus, the circuit of the adder and reg-